

Part 1: Number Systems & Assembly Conversions

1. **How do you convert a decimal number to hexadecimal manually?** Divide the decimal number by 16 repeatedly and record the remainders. Read the remainders from the last one obtained to the first to get the hex value.
2. **How do you convert a binary number to decimal?** Multiply each bit (0 or 1) by its place value (2^0 , 2^1 , 2^2 , etc.) starting from the right. Add all the results together.
3. **How do you convert a hexadecimal number to binary?** Replace each hexadecimal digit with its equivalent 4-bit binary group (e.g., A = 1010, F = 1111). Combine them to form the binary string.
4. **What is the difference between BCD and binary representation?** Binary represents the mathematical value of the whole number in base-2. BCD (Binary Coded Decimal) represents each individual decimal digit (0-9) separately using 4 bits.
5. **How can ASCII characters be converted to numeric values in assembly?** Subtract `30h` (decimal 48) from the ASCII character. For example, ASCII '5' (35h) minus 30h equals the integer 5.
6. **How do you store a decimal number in a register?** Use the `MOV` instruction followed by the register and value. For example: `MOV AX, 10` puts the decimal value 10 into the AX register.
7. **What instruction is used for arithmetic conversion in 8086?** `CBW` (Convert Byte to Word) is used to extend a signed byte into a word. Instructions like `AAA` and `DAA` are used to adjust formats after arithmetic.
8. **How do you convert a number from lower case to upper case using ASCII values?** Subtract `20h` (decimal 32) from the lowercase letter's ASCII value. For example, 'a' (61h) minus 20h becomes 'A' (41h).
9. **Explain signed vs unsigned conversion.** Unsigned conversion treats all bits as the magnitude (positive numbers only). Signed conversion checks the Most Significant Bit (MSB); if it is 1, the number is treated as negative.
10. **How do you represent negative numbers in 8086?** They are represented using **2's Complement**. To find this, you invert all bits of the positive number and add 1.
11. **How do you convert an ASCII digit to its integer value in 8086?** (Same as question 5) You subtract `30h` from the ASCII value.
12. **How is hexadecimal used in memory addressing?** Hex is used because it is shorter and easier to read than binary. Addresses are usually written as `Segment:Offset` in hex (e.g., `0700:1234`).

13. **What is the use of AAA instruction?** AAA (ASCII Adjust After Addition) corrects the result in the AL register after adding two unpacked BCD numbers so the result remains a valid decimal digit.
14. **Explain DAA instruction for BCD adjustment.** DAA (Decimal Adjust Accumulator) corrects the result of an addition to ensure the answer is a valid Packed BCD number (two decimal digits in one byte).
15. **How do you handle overflow during conversion?** Check the **Overflow Flag (OF)** in the flags register. If OF=1, the result is too big to fit in the destination register.

Part 2: EMU8086 Emulator & Operations

1. **How do you install EMU8086 on Windows?** Download the installer (setup.exe), run it, and follow the simple on-screen "Next" prompts. It installs like any standard Windows program.
2. **What are the system requirements for EMU8086?** It is very lightweight. It runs on almost any Windows version (XP to 11) and requires minimal RAM and disk space.
3. **How do you configure DOSBox for assembly programming?** EMU8086 has an *internal* emulator, so no config is needed. If using external DOSBox, you use the `mount` command to link a folder on your PC to a virtual drive letter (like C:).
4. **What is the difference between MASM and EMU8086?** MASM is a professional assembler by Microsoft. EMU8086 is an educational tool that combines an editor, assembler, and visual debugger all in one easy interface.
5. **How do you create a new assembly program in EMU8086?** Open EMU8086, click the "New" button, and select a template (like "COM template" or "EXE template").
6. **What file extension is used for 8086 assembly programs?** The source code files are saved with the `.asm` extension.
7. **How do you assemble and run a program in EMU8086?** Click the "Emulate" button to assemble the code. A new window pops up; click "Run" in that window to execute it.
8. **What is the difference between assemble and compile in EMU8086?** "Assemble" translates assembly code directly to machine code. "Compile" usually refers to saving the final standalone executable file (`.exe` or `.com`).
9. **How do you enable keyboard interrupts in EMU8086?** You don't need a specific setting. In your code, you simply call interrupt `INT 21h` with Function `01h` (read char) or `0Ah` (read string).
10. **How do you debug a program in EMU8086?** Click "Emulate," then use the "Single Step" button. This executes one line at a time so you can watch registers change value.
11. **How do you set memory segment size in EMU8086?** This is usually handled automatically by the templates (like `#MAKE_COM#`). You don't typically set this manually for

basic exams.

12. **What is the difference between real mode and protected mode in DOS? Real Mode** (8086 default) gives direct access to hardware but has limited memory (1MB). **Protected Mode** prevents direct hardware access to stop programs from crashing the system.
13. **How do you save an assembled program in EMU8086?** In the main editor, click "Save". To create a runnable program file, click "Compile" to generate a `.com` or `.exe` file.
14. **How do you open an existing program in EMU8086?** Click **File > Open** in the menu bar and select your `.asm` file.
15. **How do you view registers and memory contents during execution?** When you click "Emulate," the emulator window opens. The registers (AX, BX, etc.) are listed on the left, and memory is shown in a grid on the right.

Part 3: Registers

1. **Name all general-purpose registers in 8086.** AX, BX, CX, DX.
2. **What is the difference between 8-bit and 16-bit registers?** A 16-bit register (like AX) holds 2 bytes. It can be split into two 8-bit registers (AH and AL) that hold 1 byte each.
3. **What is the purpose of the AX register?** It is the **Accumulator**, used for most arithmetic, logic, and data transfer operations.
4. **Explain the function of the BX register.** It is the **Base** register, often used to hold the base address for memory access.
5. **What is the purpose of the CX register?** It is the **Count** register, used automatically as a counter for loops and string operations.
6. **What is the purpose of the DX register?** It is the **Data** register, used for I/O port access and to hold the upper half of large numbers in multiplication/division.
7. **Name the segment registers and their use.** CS (Code), DS (Data), SS (Stack), ES (Extra). They point to the starting addresses of the memory segments where code, data, and stack are stored.
8. **What is the difference between SP and BP?** SP (Stack Pointer) automatically points to the top of the stack. BP (Base Pointer) is used to manually access data *within* the stack (like parameters).
9. **What is the function of SI and DI registers?** SI (Source Index) and DI (Destination Index) are used mainly in string operations to point to source and destination data.
10. **Explain the role of the FLAGS register.** It contains status bits (Zero, Carry, Overflow, etc.) that indicate the result or status of the most recent operation.

11. **Which register is used for string operations?** SI and DI (Source and Destination Indexes) are the primary registers.
12. **How is the IP register used during execution?** IP (Instruction Pointer) holds the offset address of the *next* instruction the CPU needs to execute.
13. **Which registers can be used for pointer arithmetic?** BX, BP, SI, and DI are the index and base registers used for memory addressing.
14. **How is the AX register split into AH and AL?** The top 8 bits are addressed as AH (High), and the bottom 8 bits are addressed as AL (Low).
15. **What is the importance of the CS register?** CS (Code Segment) points to the memory section where the executable code lives. Without it, the CPU cannot fetch instructions.

Part 4: Arithmetic Instructions

1. **Name the basic arithmetic instructions in 8086.** ADD, SUB, MUL, DIV, INC, DEC.
2. **How does the ADD instruction work?** `ADD dest, source` adds the source value to the destination value and stores the result in the destination.
3. **How does the SUB instruction work?** `SUB dest, source` subtracts the source value from the destination value and stores the result in the destination.
4. **What is the difference between INC and ADD?** INC adds exactly 1 and does *not* affect the Carry Flag. ADD adds any value and *does* affect the Carry Flag.
5. **Explain MUL and IMUL instructions.** MUL is for **unsigned** multiplication. IMUL is for **signed** (integer) multiplication.
6. **How does DIV work?** It divides the accumulator by the source. The Quotient is stored in AL (or AX), and the Remainder is stored in AH (or DX).
7. **How do you handle carry using ADC?** Use `ADC` (Add with Carry). It adds the two numbers plus the value of the Carry Flag (1 or 0) from the previous operation.
8. **How do you handle borrow using SBB?** Use `SBB` (Subtract with Borrow). It subtracts the source and the Carry Flag (borrow) from the destination.
9. **Which flag registers are affected by arithmetic operations?** Carry (CF), Zero (ZF), Sign (SF), Overflow (OF), Parity (PF), and Auxiliary Carry (AF).
10. **How is NEG instruction used?** `NEG` finds the 2's complement of a value (effectively multiplying by -1) to change its sign.
11. **How can you perform subtraction using addition?** You `NEG` (negate) the number you want to subtract, and then `ADD` it to the other number.
12. **Explain the difference between signed and unsigned multiplication.** Unsigned multiplication treats the numbers as raw magnitude. Signed multiplication preserves the

sign bit so negative numbers are calculated correctly.

13. **How is overflow detected in arithmetic operations?** If the result is too large for the destination register (changing the sign bit incorrectly), the **Overflow Flag (OF)** is set to 1.
14. **How do you add two 16-bit numbers using 8-bit registers?** First `ADD` the lower 8 bits (AL). Then `ADC` (Add with Carry) the upper 8 bits (AH) to include any carry from the first addition.
15. **How do you perform BCD addition?** Perform a normal `ADD` instruction, and then immediately use the `DAA` (Decimal Adjust Accumulator) instruction to correct the result into valid BCD.

Part 5: Addressing Modes

1. **What is immediate addressing? Give an example.** The data (operand) is part of the instruction itself. Example: `MOV AX, 5` (5 is the immediate value).
2. **What is register addressing? Give an example.** The data is stored in a register. Example: `MOV AX, BX` (moves data from BX to AX).
3. **What is direct addressing?** You provide the exact memory address (or variable name) where the data is stored. Example: `MOV AX, [1234h]`.
4. **Explain indirect addressing mode.** The memory address is not given directly but is stored inside a register (like BX, SI, or DI).
5. **What is register indirect addressing?** You access memory using an address held in a base or index register. Example: `MOV AX, [BX]`.
6. **What is based addressing mode?** Uses a base register (BX or BP) plus a displacement (offset) to calculate the address. Useful for accessing data structures.
7. **What is indexed addressing mode?** Uses an index register (SI or DI) plus a displacement. Useful for accessing array elements.
8. **What is based plus index addressing mode?** Combines a base register (BX/BP) and an index register (SI/DI) to calculate the address. Example: `MOV AX, [BX+SI]`.
9. **How do you access memory using SI and DI?** Put the address in the register and use brackets. Example: `MOV AL, [SI]` moves the byte at the address in SI to AL.
10. **Which addressing mode is fastest? Register addressing** is fastest because the data is already inside the CPU and no memory access is required.
11. **How is segment:offset used in addressing?** The physical address is calculated as `Segment * 10h + Offset`. The segment register (like DS) provides the base, and the instruction provides the offset.

12. **Explain relative addressing.** Used in jump instructions (like `JMP`). The operand is an offset relative to the current Instruction Pointer (IP) location.
13. **How do you calculate effective address in 8086?** Effective Address (EA) = Base + Index + Displacement. The CPU calculates this automatically based on the addressing mode used.
14. **Which addressing mode is used for arrays?** Indexed addressing (using SI or DI) is typically used to iterate through arrays.
15. **Can immediate data be used in memory operations?** Yes, but you must specify the size. Example: `MOV [BX], 5` is invalid, but `MOV BYTE PTR [BX], 5` is valid.

Part 6: Display a Message

1. **Which DOS interrupt is used to display a character?** `INT 21h` is the interrupt. Specifically, you use Function `02h`.
2. **How do you display a string using INT 21h?** Set `AH = 09h`, load the address of the string into `DX` (`LEA DX, msg`), and call `INT 21h`.
3. **Which register stores the ASCII code for a character?** For printing a single character (Function `02h`), the ASCII code goes in the **DL** register.
4. **How do you display a newline in DOS?** You must print two characters: Carriage Return (ASCII 13) and Line Feed (ASCII 10).
5. **Which function of INT 21h is used for printing a string?** Function `09h` (Put `09h` in `AH`).
6. **How do you terminate a program after displaying a message?** Use `INT 21h` with Function `4Ch`. (`MOV AH, 4Ch` then `INT 21h`).
7. **How do you store a string in memory?** Define it in the data segment using `DB` (Define Byte). Example: `msg DB 'Hello$'`.
8. **How do you print a message character by character?** Use a loop that loads a character into `DL`, calls Function `02h` (`INT 21h`), and repeats until the string ends.
9. **Can you use loops to display a string?** Yes. You can loop through the string's memory address, printing one character at a time.
10. **How do you handle \$ in string printing with DOS?** For Function `09h`, the `$` symbol marks the **end** of the string. It is not printed.
11. **How do you display numeric values as characters?** You must convert the number to ASCII by adding `30h` (for digits 0-9) before printing.
12. **Which register holds the segment of a string for printing?** The **DS** (Data Segment) register must point to the segment where the string is stored.

13. **How do you print multiple messages sequentially?** Print the first message (Load DX, INT 21h), then load the address of the second message into DX and call INT 21h again.
14. **How can you display a message in uppercase only?** You must convert the characters in memory (subtract 20h from lowercase ASCII) before calling the print interrupt.
15. **How is a message terminated in memory for DOS printing?** It must end with the \$ character if you are using Function 09h.

Part 7: Logic Operators

1. **Name the basic logic instructions in 8086.** AND, OR, XOR, NOT, TEST.
2. **How does AND work in 8086?** It performs a bitwise AND. The result bit is 1 only if *both* operand bits are 1. It is often used to clear specific bits.
3. **How does OR work?** It performs a bitwise OR. The result bit is 1 if *at least one* of the operand bits is 1. It is often used to set specific bits.
4. **How does XOR work?** It performs a bitwise Exclusive OR. The result bit is 1 if the operand bits are *different*. It is used to toggle bits or clear a register.
5. **How is NOT used?** It inverts every bit in the operand (0 becomes 1, 1 becomes 0). This is also known as the 1's Complement.
6. **Which flags are affected by logic operations?** Sign (SF), Zero (ZF), and Parity (PF) are updated. Carry (CF) and Overflow (OF) are always cleared to 0.
7. **How can you use AND to mask bits?** AND the value with a binary mask where 0s correspond to bits you want to clear (remove) and 1s correspond to bits you want to keep.
8. **How can you toggle bits using XOR?** XOR the value with 1 at the positions you want to flip. $1 \text{ XOR } 1 = 0$ and $0 \text{ XOR } 1 = 1$.
9. **How can you check a specific bit using AND?** AND the value with a mask that has a 1 only at that bit's position. If the Zero Flag (ZF) is set, the bit was 0.
10. **How is SHL used in logic operations?** SHL (Shift Left) shifts bits to the left, filling with 0. It is equivalent to multiplying by 2.
11. **How is SHR used in logic operations?** SHR (Shift Right) shifts bits to the right, filling with 0. It is equivalent to dividing by 2 (unsigned).
12. **How do you perform bitwise inversion?** Use the NOT instruction on the register or memory location.
13. **How do you combine multiple logic operations?** Execute them sequentially. For example, you might AND to clear bits, then OR to set new values.
14. **How can you set or reset a bit in a register?** To **set** (make 1), use OR with a 1 mask. To **reset** (make 0), use AND with a 0 mask at that position.

15. **How is logical comparison done using CMP?** `CMP` subtracts the source from the destination but discards the result, only updating the flags (like ZF for equality) to be checked by conditional jumps.

Part 8: Arrays

1. **How do you declare an array in 8086?** Use `DB` (Define Byte) or `DW` (Define Word) followed by multiple values separated by commas. Example: `nums DB 1, 2, 3, 4`.
2. **How do you access array elements?** Load the array's starting address into a register (like SI) and access memory using that pointer. Example: `MOV AL, [SI]`.
3. **Which registers are used for array indexing?** SI (Source Index) and DI (Destination Index) are the standard registers for moving through arrays, often combined with BX.
4. **How do you loop through an array?** Set `CX` to the array size. Inside the loop, access the element, increment the pointer (SI/DI), and let the `LOOP` instruction decrement CX.
5. **How is SI used for array traversal?** `LEA SI, array` loads the start address. `[SI]` accesses the current item. `INC SI` moves to the next byte (or `ADD SI, 2` for words).
6. **How is DI used for destination indexing?** It works like SI but is typically used as the pointer for where you are writing data to (like copying an array).
7. **How do you sum all elements of an array?** Initialize a sum register (e.g., AX=0). Loop through the array adding each element to AX: `ADD AL, [SI]`.
8. **How do you find the largest element of an array?** Load the first element as "max". Loop through the rest; use `CMP` to compare current element with max. If greater, update max.
9. **How do you store array elements in memory?** They are stored in contiguous (sequential) memory addresses in the Data Segment.
10. **Can arrays be multidimensional?** Memory is linear (1D). You simulate 2D arrays by calculating the offset: `Address = Base + (Row * RowSize) + Col`.
11. **How do you reverse an array in 8086?** Use two pointers: SI at the start, DI at the end. Swap the values at `[SI]` and `[DI]`, increment SI, decrement DI, and repeat until they meet.
12. **How do you copy one array to another?** Set SI to source, DI to destination, CX to size. Loop: Read byte from `[SI]`, write to `[DI]`, increment both.
13. **How do you initialize an array with constants?** Use the `DUP` operator. Example: `arr DB 10 DUP(0)` creates an array of 10 bytes, all initialized to 0.
14. **Which addressing mode is used for arrays?** Indexed Addressing (e.g., `[SI]`) or Based Indexed (e.g., `[BX+SI]`) are most common.

15. **How do you handle array overflow?** Assembly does not check array bounds automatically. You must manually ensure your index/pointer does not go past the end of the allocated memory.

Part 9: Loops

1. **Which instructions are used for loops in 8086?** The main ones are `LOOP`, `LOOPE` (Loop if Equal), and `LOOPNE` (Loop if Not Equal).
2. **How does `LOOP` instruction work?** It automatically decrements the `CX` register by 1. If `CX` is not zero, it jumps back to the specified label; otherwise, it moves to the next line.
3. **How do you implement a counting loop?** Move the count into `CX`, write a label at the start of the code block, and place the `LOOP` instruction pointing to that label at the end.
4. **How do you implement an infinite loop?** Create a label and use an unconditional `JMP` to jump back to it repeatedly. Example: `Start: ... JMP Start`.
5. **How do you implement a loop with `JCXZ`?** `JCXZ` (Jump if CX Zero) is used at the very start to skip the loop entirely if `CX` is already 0, preventing errors.
6. **Which register controls the number of iterations?** The `CX` (Count) register is the dedicated loop counter.
7. **How do you nest loops in 8086?** Push the outer loop's `CX` value onto the stack (`PUSH CX`) before starting the inner loop, and pop it back (`POP CX`) after the inner loop finishes.
8. **How can loops be used to traverse arrays?** Set `CX` to the array length, and use `SI` or `DI` to index elements. Inside the loop, increment the index register to move to the next item.
9. **How do you decrement a counter in a loop?** The `LOOP` instruction decrements `CX` automatically. If using a custom counter, you must manually use `DEC register`.
10. **How do you terminate a loop on a condition?** Use a `CMP` instruction inside the loop followed by a conditional jump (like `JE exit_label`) to break out early.
11. **How do you use `CMP` inside a loop?** Place `CMP` inside the loop body to check a value. If the condition is met, use a jump instruction to exit or skip code.
12. **How do you use `JMP` to create a loop?** Write a label, execute your code, modify a counter/check condition, and use `JMP` to go back to the label if the loop should continue.
13. **How do loops affect the `CX` register?** Every time the `LOOP` instruction executes, the value in `CX` decreases by exactly 1.
14. **How do you implement a sum of numbers using a loop?** Initialize a sum register (like `AX`) to 0. Loop `CX` times, adding the current number to `AX` on each iteration.

15. **How do you implement a factorial using a loop?** Initialize `AX` to 1 and `CX` to the number (e.g., 5). Loop, multiplying `AX` by `CX`, until `CX` becomes 1.

Part 10: Jumps

1. **What is the difference between conditional and unconditional jumps?** Unconditional jumps (`JMP`) always happen. Conditional jumps (like `JE`) only happen if specific Flags (Zero, Carry, etc.) are set.
2. **Which instruction is used for unconditional jump?** The `JMP` instruction.
3. **Name conditional jump instructions.** `JE` (Equal), `JNE` (Not Equal), `JG` (Greater), `JL` (Less), `JC` (Carry), `JZ` (Zero), `JS` (Sign).
4. **How does `JE/JZ` work?** It jumps if the Zero Flag (ZF) is 1, meaning the previous comparison found the two values were equal.
5. **How does `JNE/JNZ` work?** It jumps if the Zero Flag (ZF) is 0, meaning the values were not equal.
6. **How does `JC` work?** It jumps if the Carry Flag (CF) is 1. Used to handle unsigned overflow or borrows.
7. **How does `JNC` work?** It jumps if the Carry Flag (CF) is 0 (No Carry).
8. **How does `JL/JNGE` work?** It jumps if the destination was **Less** than the source (Signed comparison, checks Sign and Overflow flags).
9. **How does `JG/JNLE` work?** It jumps if the destination was **Greater** than the source (Signed comparison).
10. **How does `JO/JNO` work?** It jumps if the Overflow Flag (OF) is set (`JO`) or not set (`JNO`). Used to detect signed arithmetic errors.
11. **How do jumps work with loops?** They are used to create custom loops (Jump back to start) or to break out of loops early (Jump to end).
12. **How do you implement a simple decision using jump?** Use `CMP` to compare values. If the condition is true, jump to the "True" block; otherwise, let the code fall through to the "False" block.
13. **How do you implement a nested if using jumps?** If the first check passes, jump to a label that contains the second `CMP` and jump. If any check fails, jump to the end.
14. **Which flag registers are checked during jumps?** The Status Flags: Zero (ZF), Carry (CF), Sign (SF), Overflow (OF), and Parity (PF).
15. **How do jumps affect program flow?** They change the value of the **IP** (Instruction Pointer) register, causing the CPU to fetch the next instruction from a different memory address.

Part 11: Bubble Sort

1. **What is bubble sort algorithm?** It is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
2. **How do you implement bubble sort in 8086?** Use nested loops. The outer loop controls the number of passes, and the inner loop performs the comparisons and swaps.
3. **Which registers are used as counters in bubble sort?** `CX` is typically used for the outer loop count. `DX` or a saved `CX` value is often used for the inner loop count.
4. **How do you compare two adjacent array elements?** Load one element into a register (e.g., `MOV AL, [SI]`) and compare it with the next memory location (`CMP AL, [SI+1]`).
5. **How do you swap elements in 8086?** Use the `XCHG` instruction (e.g., `XCHG AL, [SI+1]`) or use a temporary register to hold one value while moving the other.
6. **How do loops work in bubble sort?** The outer loop runs $N - 1$ times. The inner loop runs through the unsorted portion of the array, comparing neighbors.
7. **How do you optimize bubble sort for fewer passes?** Use a flag (register set to 0 or 1). If a pass completes with **no swaps**, the array is already sorted, and you can stop early.
8. **How do you detect if array is already sorted?** If the inner loop finishes an entire pass without performing a single swap operation.
9. **How do you store the sorted array back in memory?** Bubble sort is an "in-place" algorithm. The swapping happens directly in the memory where the array is stored, so no extra step is needed.
10. **Which addressing mode is used for array elements?** **Indexed Addressing** (using `SI`) is the standard way to point to current elements during the loop.
11. **How do you handle multiple passes in bubble sort?** At the start of every outer loop iteration, you must **reset the pointer** (`SI`) to the beginning of the array.
12. **How is `CX` register used in outer loop?** It holds the count of passes ($\text{Length} - 1$). The `LOOP` instruction at the end of the outer block decrements it.
13. **How is inner loop implemented?** It compares elements `[SI]` and `[SI+1]`. If they are out of order, it swaps. Then it increments `SI` and decrements the inner counter.
14. **How do you increment pointers to traverse the array?** Use `INC SI` if sorting Bytes. Use `ADD SI, 2` if sorting Words (16-bit numbers).
15. **How do you handle array of bytes vs words?** For bytes, use `AL` and increment `SI` by 1. For words, use `AX`, compare with `[SI+2]`, and increment `SI` by 2.

Part 12: Shifts

1. **What is the difference between logical and arithmetic shift?** Logical shift fills empty bits with 0. Arithmetic shift preserves the sign bit (MSB) to handle signed numbers correctly.
2. **How does SHL work? Shift Left:** It moves bits to the left. The LSB is filled with 0, and the MSB moves into the Carry Flag.
3. **How does SHR work? Shift Right:** It moves bits to the right. The MSB is filled with 0, and the LSB moves into the Carry Flag.
4. **How does SAL differ from SHL?** In 8086, `SAL` (Shift Arithmetic Left) and `SHL` are the **exact same instruction** and generate the same machine code.
5. **How does SAR work for signed numbers? Shift Arithmetic Right:** It shifts right but **copies the original Sign Bit** into the new vacant MSB position to keep the number positive/negative.
6. **How many bits can you shift in one instruction?** You can shift by **1** directly (e.g., `SHL AX, 1`) or by `CL` for multiple bits (e.g., `MOV CL, 3; SHL AX, CL`).
7. **How do shifts affect carry flag?** The last bit that is "shifted out" of the register falls into the **Carry Flag (CF)**.
8. **How do you multiply a number by 2 using shift?** Use `SHL` (Shift Left) by 1 position.
9. **How do you divide a number by 2 using shift?** Use `SHR` (Shift Right) for unsigned numbers, or `SAR` (Shift Arithmetic Right) for signed numbers.
10. **How do you shift multi-bit numbers in registers?** Shift the lower word first, then use `RCL` (Rotate through Carry) or `RCR` on the upper word to pull in the bit shifted out of the lower word.
11. **How do you implement bit masking using shift?** Shift the number to align the bits you want, then use `AND` to isolate them (masking out the others).
12. **How is ROL different from SHL? Rotate Left (ROL)** takes the bit falling off the left side and puts it back into the right side (circular). SHL just throws it away (into Carry).
13. **How is ROR different from SHR? Rotate Right (ROR)** takes the bit falling off the right side and puts it back into the left side. SHR fills with 0.
14. **Can you shift memory contents directly?** Yes. Example: `SHL BYTE PTR [BX], 1` shifts the byte at the memory address in BX.
15. **How do shifts help in bit manipulation?** They allow you to access specific bits, pack/unpack data, and perform very fast multiplication and division by powers of 2.

Part 13: Rotation

1. **What is rotation in 8086?** It shifts bits left or right, but unlike standard shifts, bits falling off one end are put back onto the other end or into the carry flag.
2. **How does ROL work? Rotate Left:** Shifts bits left; the MSB moves into the LSB position *and* the Carry Flag.
3. **How does ROR work? Rotate Right:** Shifts bits right; the LSB moves into the MSB position *and* the Carry Flag.
4. **How does RCL differ from ROL? Rotate Carry Left:** It treats the Carry Flag as part of the circle, rotating bits through the Carry Flag (9 bits for byte, 17 bits for word).
5. **How does RCR differ from ROR? Rotate Carry Right:** It rotates bits through the Carry Flag, acting as an extension of the register.
6. **How do carry flags affect rotation?** In `RCL / RCR`, the Carry Flag is part of the loop. In `ROL / ROR`, the Carry Flag just stores a copy of the bit that was rotated.
7. **How do you rotate multiple bits?** Move the count into `CL` (e.g., `MOV CL, 4`) and then use the instruction (e.g., `ROL AX, CL`).
8. **How do you use rotation for encryption?** Rotating bits rearranges them in a way that scrambles data, which can be part of a simple encryption algorithm.
9. **How do you rotate a byte stored in memory?** Use the `PTR` directive to specify size. Example: `ROL BYTE PTR [SI], 1`.
10. **How do you rotate 16-bit numbers?** Use a 16-bit register like `AX`. Example: `ROR AX, 1`.
11. **How is rotation different from shift?** Shift instructions throw away bits that fall off (or store them only in Carry), while rotation instructions recycle them back into the number.
12. **How do you rotate circularly?** `ROL` and `ROR` are circular rotations by definition—no bits are lost.
13. **How do you rotate signed numbers?** Rotation treats data as raw bits, so it ignores signs. You just rotate the bit pattern regardless of what it represents.
14. **How do rotations affect high and low bits?** In `ROL`, high bits wrap around to low positions. In `ROR`, low bits wrap around to high positions.
15. **How do you use rotation in masking operations?** You can rotate a bit of interest into the LSB or MSB position to easily test it or isolate it.

Part 14: Subroutine

1. **What is a subroutine in 8086?** It is a reusable block of code (like a function) that performs a specific task and can be called from anywhere in the main program.
2. **How do you define a subroutine?** Use a label followed by `PROC` (procedure) and end it with `RET` and `ENDP`.

3. **How do you call a subroutine?** Use the `CALL` instruction followed by the subroutine name (e.g., `CALL MyProc`).
4. **How do you return from a subroutine?** Use the `RET` (Return) instruction at the end of the subroutine.
5. **How do you pass parameters to a subroutine?** You can put values in registers (fastest) or push them onto the stack before calling.
6. **Which register is used for return address?** The return address is stored on the **Stack**, not in a specific general-purpose register.
7. **How is the stack used in subroutines?** The `CALL` instruction pushes the return address (IP) onto the stack, and `RET` pops it back to resume execution.
8. **How do local variables work in subroutines?** You can reserve space on the stack by subtracting from `SP` (Stack Pointer) or using `BP` (Base Pointer) to access that stack memory.
9. **Can a subroutine call another subroutine?** Yes, this is called nesting. The stack automatically manages the return addresses for each level.
10. **How do you implement recursion?** A subroutine calls itself. You must have a condition to stop calling (base case) so the stack doesn't overflow.
11. **How do flags behave in subroutines?** Subroutines generally affect flags just like any other code. If you need to preserve flags, `PUSHF` (Push Flags) and `POPF` (Pop Flags).
12. **How do you save registers before a subroutine call?** `PUSH` the registers you want to save onto the stack at the start of the subroutine.
13. **How do you restore registers after return?** `POP` the registers from the stack in the **reverse order** they were pushed just before the `RET` instruction.
14. **How do you use subroutines for code reuse?** Write the code once (e.g., a print function), and `CALL` it multiple times whenever needed instead of rewriting the instructions.
15. **How do you handle multiple return points?** It's best to have one exit label with the `RET` instruction and jump to it from other points in the subroutine.

Part 15: Subroutine with Parameters

1. **How do you pass parameters through registers?** Simply `MOV` data into registers (like `AX`, `BX`) before the `CALL`. The subroutine reads those registers.
2. **How do you pass parameters through stack?** `PUSH` the values onto the stack before the `CALL`. Inside the subroutine, access them using `BP` relative to the stack frame.
3. **How do you access parameters inside subroutine?** If passed via stack: `MOV BP, SP` and access like `[BP+4]`. If via registers: just use the register names.

4. **How do you return values from subroutine?** Typically, place the result in the `AX` register before executing `RET`.
5. **Which registers are commonly used for parameters?** `AX`, `BX`, `CX`, and `DX` are common.
6. **How do you pass arrays as parameters?** Pass the **address** (offset) of the array in a register like `SI` or `BX` (`LEA SI, MyArray`).
7. **How do you pass byte vs word parameters?** Just ensure you use the correct register size (e.g., `AL` for byte, `AX` for word) or `PUSH` the correct size.
8. **How do you increment pointer to access multiple parameters?** If passing an array address in `SI`, use `INC SI` (for bytes) or `ADD SI, 2` (for words) inside the loop.
9. **How do you implement subroutine with conditional execution?** Pass a flag or value. Inside the subroutine, use `CMP` and jumps to decide which parts to run.
10. **How do you handle nested subroutine calls with parameters?** Each subroutine should protect its own registers (`PUSH/POP`) so inner calls don't overwrite the outer caller's data.
11. **How do you preserve registers while using parameters?** `PUSH` any registers you intend to change at the start of the procedure and `POP` them at the end.
12. **How do you pass constants as parameters?** Move the immediate value into a register (`MOV AX, 5`) before calling.